Intermediate (.env by hand) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Where this exercise is saved: `practice/env_config_practice.py`, under its `# Intermediate` section — the same file also holds the Edge cases exercise, under an `# Edge cases` section. Run it with `cd practice && python env_config_practice.py`.

Used later by: the Build Task's `load_config()` in `practice/build_task/config.py` — it ships the short `load_dotenv()` + `os.getenv()` version you land on here, re-written (not imported) so it can return a typed `Config` object.

All examples below assume a `.env` file next to your script — `practice/.env` — containing:

```
# practice/.env
# credentials for the API client
OPENAI_API_KEY="sk-test123"
LOG_LEVEL=DEBUG
```

Story — `env_config_practice.py` (Intermediate section): before trusting `python-dotenv` to read a `.env` file, this exercise writes the same parsing loop by hand once, so the library stops being a black box. **If not:** you'd never see why quoting, blank lines, and comments actually need special handling — and `load_dotenv()` would feel like magic instead of "oh, that's just a loop with a dictionary."

Solution

```
# practice/env_config_practice.py
def parse_env_by_hand(path: str) -> dict[str, str]:
    env_vars: dict[str, str] = {}
    for line in open(path):
        line = line.strip()
        if not line or "=" not in line:
            continue
        key, value = line.split("=", 1)
        env_vars[key] = value
    return env_vars

values = parse_env_by_hand(".env")
```

```
print(values["OPENAI_API_KEY"])
print(values["LOG_LEVEL"])
```

Expected output:

```
"sk-test123"
DEBUG
```